TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision under Hard Constraints and Multimodal Uncertainty

Anonymous Author(s)

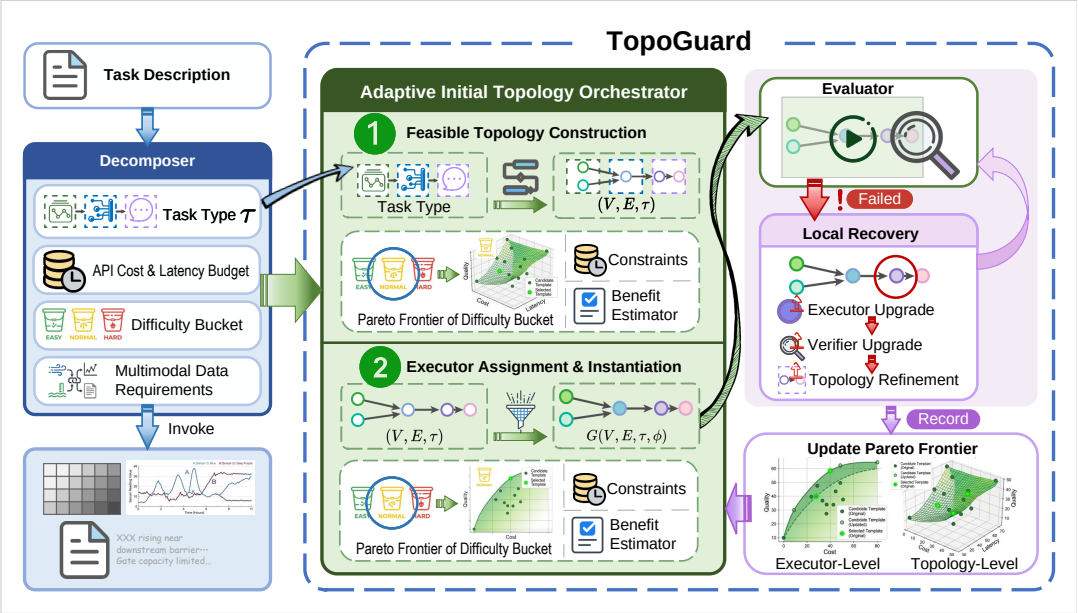

Figure 1: Overview of the TopoGuard framework. The system takes a task description, resource constraints, and multimodal inputs, constructs an initial executable workflow through topology-level and executor-level Pareto-guided selection, monitors execution with an evaluator, and performs bounded local recovery through executor upgrade, verifier upgrade, or topology refinement when failures occur.

Abstract

Risk-sensitive decision systems, such as storm surge forecasting, increasingly rely on multi-stage collaboration among heterogeneous agents and multimodal evidence. However, such systems face a dual challenge: multimodal uncertainty (e.g., noisy sensors or conflicting weather patterns) destabilizes evidence reliability, while hard constraints (e.g., safety thresholds, strict time windows, and mandatory human oversight) prohibit unconstrained online exploration. Fixed pipelines are brittle under uncertainty, yet trial-and-error is infeasible under strict safety boundaries. We propose **TopoGuard**, an adaptive topology orchestration framework designed to construct feasible and reliable execution workflows. Instead of exhaustive online search, TopoGuard maintains task-conditional Accuracy–Cost (ACC–Cost) profiles for candidate agent combinations, initialized from a small calibration set. These profiles guide the rapid generation of a high-quality initial topology aligned with current risk levels and constraints. During execution, evaluator modules monitor both output quality and constraint satisfaction, providing structured feedback for periodic profile recalibration. When failures occur, TopoGuard performs local repair by upgrading only affected stages, ensuring system robustness without full replanning. Experimental results in water conservancy digital twins demonstrate that TopoGuard significantly reduces computational costs while maintaining high-fidelity reliability under extreme uncertainty.

CCS Concepts

• **Information systems** → **Decision support systems**; **Multimedia information systems**; • **Computing methodologies** → *Artificial intelligence*; *Planning and scheduling*.

Keywords

Multimedia Systems, Topology Orchestration, Multimodal Decision Support, Workflow Optimization, Local Graph Repair, Digital Twin

ACM Reference Format:

Anonymous Author(s). 2026. TopoGuard: Adaptive Topology Orchestration for Risk-Sensitive Decision under Hard Constraints and Multimodal Uncertainty . In . ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Modern multimodal intelligent systems are increasingly used for decision support in risk-sensitive domains such as hydrological analysis, infrastructure monitoring, and emergency response [1, 3, 7, 20]. In these settings, task completion typically requires the coordinated use of heterogeneous functional modules, including

data retrieval, numerical computation, reasoning, and verification [6, 26, 27]. System effectiveness therefore depends not only on individual executors, but also on how they are organized into an executable collaboration topology.

In practice, many systems still rely on manually designed workflows or rigid model-selection policies where functional nodes and execution order are predetermined [5, 12, 25, 28]. Such designs are easy to deploy but brittle in uncertain environments: different task instances require different processing structures, intermediate outputs may fail, and execution must satisfy constraints on quality, cost, and latency. A workflow that performs well for one instance may become expensive, slow, or ineffective for another, especially in risk-sensitive applications where local failures can propagate into unreliable downstream decisions [19]. This issue is further amplified in multimodal settings with missing or unreliable modalities [9, 13, 14, 17, 21].

Existing approaches attempt to improve adaptability through model routing or dynamic tool selection [8, 18, 23]. However, the core decision is not only which executor to use, but also which functional nodes should be activated, how they should be connected, and how the execution graph should be adjusted when failures occur. In other words, the decision variable is the workflow topology itself. Recent work on LLM planning and workflow generation highlights structured execution organization, but mainly focuses on planning quality rather than topology adaptation under runtime constraints [5, 6, 12, 26, 28]. We therefore formulate the problem as *adaptive topology orchestration* under hard constraints and multimodal uncertainty.

To address this problem, we propose **TopoGuard**, an adaptive topology orchestration framework for risk-sensitive decision systems. TopoGuard treats workflow execution as a closed-loop decision process over structured graphs. It first builds offline performance profiles for candidate executors and estimates the expected quality, cost, and latency of candidate topologies. Based on these estimates, TopoGuard performs constrained Pareto-based selection to obtain an initial executable graph. During execution, evaluator modules monitor intermediate outputs and detect local failures or reliability risks. Instead of triggering global replanning, TopoGuard performs bounded local graph repair to improve robustness while controlling additional overhead.

The key insight of this work is that robust multimodal decision systems should move beyond choosing better components to choosing and maintaining better *collaboration topologies*. Topology-level adaptation allows the execution graph to adjust to task requirements, resource constraints, and runtime uncertainty, which is essential for balancing effectiveness, efficiency, and reliability in risk-sensitive environments.

We evaluate TopoGuard on representative tasks involving heterogeneous executors and structured workflow operators. Experimental results show that TopoGuard achieves better quality-cost-latency trade-offs and stronger robustness than static baselines, particularly when task requirements vary and intermediate failures occur.

The main contributions of this paper are summarized as follows:

- **Problem Formulation.** We formulate adaptive workflow execution in risk-sensitive multimodal systems as an adaptive topology orchestration problem under hard constraints and multimodal uncertainty.
- **TopoGuard Framework.** We propose **TopoGuard**, a feasibility-aware topology orchestration framework that integrates offline performance profiling, constrained Pareto-based initial topology selection, and evaluator-triggered bounded local repair.
- **Empirical Validation.** We demonstrate that TopoGuard achieves better quality-cost-latency trade-offs and stronger robustness than static and non-adaptive alternatives in constrained multimodal environments.

2 Related Work

2.1 Multimedia Decision Systems in High-Stakes Applications

Multimedia systems are increasingly used beyond perception and retrieval, and are becoming important components in high-stakes decision-support settings such as emergency response, infrastructure monitoring, and hydrological analysis [16, 19, 24]. In these applications, system effectiveness depends not only on processing heterogeneous evidence, but also on coordinating multiple functional modules across stages. However, most existing work focuses on perception, prediction, or task-specific modeling, while relatively less attention has been paid to how such modules should be dynamically organized into executable workflows under operational constraints. Our work addresses this gap by focusing on topology-level orchestration for constrained multimodal decision systems.

2.2 Multimodal Uncertainty and Robustness

A substantial body of multimodal research studies robustness under missing, noisy, or unreliable inputs [10]. These efforts are highly relevant because real-world multimodal systems often face partial observability, degraded evidence quality, and cross-source inconsistency. Nevertheless, prior work mainly improves robustness within a fixed model or pipeline. By contrast, we study how uncertainty should affect the execution structure of the system itself, including whether different workflow topologies, additional verification stages, or bounded recovery actions are needed during execution.

2.3 Workflow Orchestration, Tool Use, and Cost-Aware Selection

Another related line of work concerns adaptive tool use, graph-based workflow execution, and efficiency-aware model selection. Planning methods such as ReAct and Tree of Thoughts demonstrate the value of adaptive intermediate decision making in LLM-based systems [22, 23]. Graph-native frameworks further show that multi-step AI processes can be represented as workflows or graphs rather than fixed linear pipelines [11, 15]. In parallel, cost-aware routing and cascading methods improve deployment efficiency by balancing quality and resource consumption [2, 4].

These directions are closely related, but they do not directly address the problem studied in this paper. Planning methods mainly

focus on reasoning trajectories or tool interaction; graph frameworks provide execution abstractions but do not explicitly optimize topology under task-dependent constraints; and routing methods typically select models or cascades for a query rather than jointly determining workflow structure, node activation, and local repair. Our work differs in treating execution topology itself as the key decision variable and formulating orchestration as a constrained multi-objective problem over workflow selection, executor assignment, and bounded local repair.

3 Method

3.1 Problem Setup

We study adaptive workflow orchestration for complex multi-stage decision tasks. Given an input task X, the system jointly optimizes: (i) workflow structure (template/topology), (ii) node-level executor assignments, and (iii) local repair actions when execution quality falls below a threshold.

The method can be summarized as hierarchical Pareto topology orchestration with local graph repair. The problem is NP-hard due to the combinatorial workflow-graph configuration space, motivating hierarchical decomposition and local repair optimization. Our main methodological contributions are: (1) hierarchical workflow-graph optimization via decomposition, (2) Pareto-based candidate pruning for combinatorial decision reduction, and (3) edit-loss-aware minimal-change local repair optimization.

3.2 Global Objective and Constraints

The overall problem is formulated as a constrained combinatorial optimization over workflow graphs:

$$\max_{\pi \in \Pi} \ \mathbb{E}_{X \sim D}\big[Q(G_\pi(X); X)\big] \tag{1}$$

subject to

$$\begin{aligned} &G_\pi(X) \in \mathrm{DAG}, \\ &C(G_\pi(X); X) \le B(X), \\ &L(G_\pi(X); X) \le \Delta(X), \\ &\mathrm{repair}(a_t): \ \mathcal{L}_{\mathrm{edit}}(a_t) \le \epsilon_{\mathrm{edit}}, \ S(G_{t+1}(a_t); X) \ge \tau_{\mathrm{pass}} \ \text{(if feasible)}. \end{aligned} \tag{2}$$

Here π is a profile-induced optimization policy, and Pareto frontiers are used only for candidate pruning.

To address scale mismatch among quality, cost, and latency, we adopt a normalized matrix utility form:

$$\mathbf{m}(G;X) = \begin{bmatrix} S(G;X) \\ C(G;X) \\ L(G;X) \end{bmatrix}, \quad \mathbf{D} = \mathrm{diag}(\sigma_S^{-1}, \sigma_C^{-1}, \sigma_L^{-1}), \quad \mathbf{R} = \mathrm{diag}(1, -1, -1), \tag{3}$$

$$\tilde{\mathbf{m}}(G;X) = \mathbf{D}\big(\mathbf{m}(G;X) - \boldsymbol{\mu}\big), \quad \mathbf{w} = \begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix}, \quad Q(G;X) = \mathbf{w}^\top \mathbf{R} \tilde{\mathbf{m}}(G;X). \tag{4}$$

When metrics are pre-normalized to comparable scales, Eq. (4) reduces to $Q = \alpha S - \beta C - \gamma L$.

3.3 Workflow Representation

A workflow is represented as a directed graph

$$G = (V, E, \tau, \rho, \phi), \tag{5}$$

where V is the node set, $E \subseteq V \times V$ is the directed edge set, $\tau(v)$ is node role type, $\rho(v)$ is executor function type, and $\phi(v)$ is runtime configuration. DAG topology is enforced by a topological order $\mathrm{ord}: V \to \{1, \ldots, |V|\}$ such that

$$(u, v) \in E \Rightarrow \mathrm{ord}(u) < \mathrm{ord}(v). \tag{6}$$

Task context is encoded into a discrete difficulty level b, which conditions workflow template retrieval and executor candidate selection. Detailed estimator and encoding forms are deferred to the appendix.

3.4 Difficulty-Aware Hierarchical Initialization

Given task X and difficulty level b, initialization is decomposed into two levels:

$$T_0 = \pi_{\mathrm{init}}^T(X, b), \qquad \Phi_0 = \pi_{\mathrm{init}}^N(X, b, T_0), \qquad G_0 = (T_0, \Phi_0). \tag{7}$$

At the template level, we first perform Pareto pruning over candidate topologies:

$$\mathcal{P}_T = \mathrm{Pareto}_{\max S, \min C, \min L}(\mathcal{T}(X, b)), \tag{8}$$

and then select the feasible template with highest proxy utility:

$$T_0 = \arg \max_{T \in \mathcal{P}_T^{\mathrm{feasible}}} \widehat{Q}_T(T; X, b). \tag{9}$$

At the node level, each executor node retrieves a difficulty-conditioned candidate pool,

$$\mathcal{A}_v(b, \rho(v)), \qquad \mathcal{P}_N(v) = \mathrm{Pareto}_{\max S, \min C}(\mathcal{A}_v(b, \rho(v))), \tag{10}$$

and selects the feasible candidate with maximal utility:

$$Q_N(a; v, X, b) = \alpha_N S(a; v, X, b) - \beta_N C(a; v, X, b), \tag{11}$$

$$\phi(v) = \arg \max_{a \in \mathcal{P}_N^{\mathrm{feasible}}(v)} Q_N(a; v, X, b). \tag{12}$$

3.5 Local Repair with Edit-Loss

During execution, node outputs are evaluated by a binary pass gate; failed nodes trigger local graph repair rather than full replanning:

$$a_t = \pi_{\mathrm{repair}}(s_t), \qquad G_{t+1} = \mathcal{T}_{\mathrm{op}}(G_t, a_t). \tag{13}$$

Repair actions are $\{A, B, C\}$, corresponding to topology/template upgrade, executor upgrade, and evaluator upgrade. We define edit-loss as

$$\mathcal{L}_{\mathrm{edit}}(a) = \lambda_n \Delta n(a) + \lambda_\phi \Delta \phi(a) + \lambda_e \Delta e(a). \tag{14}$$

Repair is then formulated as a minimal-edit local search over the workflow graph:

$$a_t^* = \arg \max_{a \in \mathcal{A}_t} \Big(Q\big(G_{t+1}(a); X\big) - \lambda\, \mathcal{L}_{\mathrm{edit}}(a)\Big), \qquad \mathcal{A}_t = \{A, B, C\}. \tag{15}$$

With hard pass feasibility,

$$\mathcal{A}_t^{\mathrm{feasible}} = \{a \in \mathcal{A}_t \mid S(G_{t+1}(a); X) \ge \tau_{\mathrm{pass}}\}, \tag{16}$$

and the repaired graph is

$$G_{t+1}^* = \mathcal{T}_{\mathrm{op}}(G_t, a_t^*). \tag{17}$$

Algorithm 1 Hierarchical Pareto Orchestration

Require: Task X, profile $\mathcal{P}$, budget Λ, deadline Δ
  $b \leftarrow \textsc{EstimateDifficulty}(X)$
  $T_0 \leftarrow \textsc{SelectTemplate}(\mathcal{P}, X, b, \Lambda, \Delta)$
  for $v \in V(T_0)$ **do**
    $\phi(v) \leftarrow \textsc{SelectExecutor}(\mathcal{P}, v, X, b, \Delta)$
  end for
  Execute workflow $G_0 = (T_0, \Phi_0)$
  if any node fails **then**
    Generate repair candidates
    Select $a^* = \arg\max(Q - \lambda L_{\text{edit}})$
    Apply repair and update workflow
  end if
  Update profiles and return workflow G^*

Candidate utility uses Eq. (4); the full repaired-utility form is provided in the appendix.

4 Experiments

We evaluate TopoGuard as a complete closed-loop topology orchestration framework. For each task instance, the system first constructs an executable workflow by jointly determining the workflow structure and node-level executors under quality–cost–latency constraints, and then monitors intermediate outputs during execution to trigger bounded local repair when necessary. Accordingly, our experiments are designed to answer the following questions:

- **RQ1:** Can TopoGuard achieve a strong overall trade-off among quality, cost, and latency?
- **RQ2:** Does local repair contribute substantively to the final system performance?
- **RQ3:** Do the observed advantages remain stable across task domains and different utility preferences?

4.1 Experimental Setup

Platform and task domains. We conduct all experiments on a water-conservancy digital-twin platform and evaluate TopoGuard on two representative task domains: **Water QA** and **Storm Surge**. The primary benchmark, **Water QA**, is formulated as a multi-stage question answering task over hydrological knowledge. Different task instances may require different execution structures, such as retrieval, reasoning, computation, verification, and aggregation. The auxiliary benchmark, **Storm Surge**, is used to examine whether the learned orchestration strategy can transfer to another decision domain without redesigning the core decision logic.

Data and evaluation protocol. For Water QA, we construct a benchmark containing training and test executions, which are further aggregated into historical performance profiles for candidate workflows and executors. For Storm Surge, we build an auxiliary transfer benchmark with heterogeneous execution choices and evaluate whether the same orchestration strategy can generalize across tasks. In both domains, the training split is used to estimate the expected quality, cost, and latency of candidate workflows, while all final results are reported on held-out test executions.

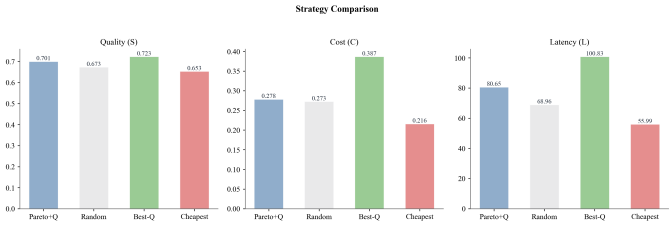

Figure 2: Strategy comparison on Water QA. TopoGuard achieves a stronger overall balance across quality, cost, and latency than the alternative orchestration strategies.

Given a test context, the system first queries historical profiles to estimate the expected triplet (S, C, L) for each candidate topology or executor, applies hard constraint filtering, computes the feasible Pareto frontier, and then selects an execution plan according to the corresponding orchestration strategy. For TopoGuard, the selected plan is further executed in a closed loop with evaluator monitoring and optional local repair.

Compared methods. We compare TopoGuard with three alternative orchestration strategies defined on the same feasible candidate pool:

- **Random**: uniformly samples a candidate from the feasible Pareto frontier;
- **Best-Quality**: selects the candidate with the highest estimated quality;
- **Cheapest**: selects the candidate with the lowest estimated cost.

These baselines represent distinct decision biases and serve as controlled references for evaluating the trade-off behavior of TopoGuard. Importantly, the comparison is performed at the topology orchestration level rather than at the level of isolated executor selection.

Metrics. We report three primary metrics: **quality** $S \in [0, 1]$, **cost** C (USD), and **latency** L (seconds). Quality measures the final task-solving effectiveness of the selected workflow. Cost is computed from actual execution expenses, and latency records the realized end-to-end execution delay. Following the current implementation, cost and latency are log-normalized only for internal Pareto filtering, while all reported results use raw values for interpretability.

Implementation details. The full TopoGuard pipeline consists of: (1) initial context analysis, (2) workflow-level candidate screening, (3) node-level executor selection, (4) execution and evaluator monitoring, and (5) bounded local repair. The repair operators currently include executor upgrade, evaluator upgrade, and workflow/topology upgrade, which together implement a closed-loop orchestration process of *structure selection* $\rightarrow$ *node selection* $\rightarrow$ *execution* $\rightarrow$ *evaluation* $\rightarrow$ *local repair*.

Table 1: Closed-loop topology orchestration results on Water QA (N=255). Best values are boldfaced.

Method	Quality S ↑	Cost (USD) ↓	Latency (s) ↓	Valid Contexts
TopoGuard	0.701	0.278	80.65	**238/255**
Random	0.673	0.273	68.96	227/255
Best-Quality	**0.723**	0.387	100.83	**238/255**
Cheapest	0.653	**0.216**	**55.99**	225/255

4.2 Main Results on Water QA

We first answer **RQ1**: whether TopoGuard can achieve a strong overall trade-off among quality, cost, and latency on the primary Water QA benchmark.

Table 1 summarizes the closed-loop results on 255 test contexts. TopoGuard achieves the strongest overall balance among the compared methods. Compared with **Random**, it improves quality from 0.673 to 0.701 with only a small increase in cost and latency. Compared with **Cheapest**, it improves quality from 0.653 to 0.701, indicating that purely cost-driven orchestration tends to sacrifice final task quality. Compared with **Best-Quality**, TopoGuard reduces cost from 0.387 to 0.278 and latency from 100.83s to 80.65s, while sacrificing only 0.022 quality points.

These results are further visualized in Fig. 2, which shows the three primary metrics separately. The figure makes the central trade-off clear: **Best-Quality** achieves the highest quality, but at substantially higher cost and latency; **Cheapest** minimizes execution overhead, but yields the lowest quality; and TopoGuard lies in a more favorable middle region, preserving strong task performance while avoiding the excessive overhead of single-preference selection. This behavior is exactly aligned with the design goal of TopoGuard, namely, to locate better operating points on the feasible frontier rather than optimizing any single metric in isolation.

4.3 Pareto Structure Analysis

To better understand *why* TopoGuard achieves the above trade-off, we further analyze the Pareto structure at both the workflow and node levels.

Fig. 3 visualizes the template-level Pareto frontiers under two objective views: quality versus cost and quality versus latency. Several candidate workflow templates form a clear frontier, ranging from low-cost direct execution to deeper structures with verification and aggregation. This figure reveals an important property of the search space: the highest-quality template is not the lowest-cost or lowest-latency one, and the feasible frontier contains multiple non-dominated choices with different system behaviors. Therefore, the orchestration problem is inherently a multi-objective selection problem at the topology level, rather than a simple one-dimensional ranking problem.

Fig. 4 further shows the node-level Pareto curves for different function types, including retrieval, reasoning, computation, verification, and aggregation. Across node types, the candidate executors also exhibit clear quality–cost trade-offs, and the shapes of the frontiers vary substantially by function. This observation justifies the two-level design of TopoGuard: the final workflow quality is determined not only by which high-level structure is selected, but also by how each node is instantiated under local trade-offs. Together, these two figures show that both workflow templates and node executors are naturally organized by Pareto structure, making hierarchical orchestration a principled solution rather than a heuristic design choice.

Table 2: Zero-shot transfer results on Storm Surge. Best values are boldfaced.

Method	Quality S ↑	Cost (USD) ↓	Latency (s) ↓
TopoGuard	**0.833**	**0.044**	18.4
Best-Quality	0.829	0.045	17.3
Random	0.823	0.046	**16.0**
Cheapest	0.807	0.046	17.5

4.4 Cross-Task Transfer to Storm Surge

We next examine **RQ3** from the cross-task perspective: whether the same orchestration framework can generalize beyond Water QA.

For this purpose, we directly apply the same orchestration logic to the Storm Surge benchmark without redesigning the decision mechanism. As shown in Table 2, TopoGuard again achieves the best quality (0.833), while maintaining cost and latency comparable to the baselines. Although the absolute performance gap is smaller than that on Water QA, the transfer result remains informative for two reasons. First, it suggests that the advantage of TopoGuard does not depend on a single task template. Second, it indicates that the proposed method captures a general orchestration principle—selecting feasible high-utility workflows under quality–cost–latency constraints—rather than overfitting to one benchmark.

4.5 Ablation on Local Repair

We now answer **RQ2**: whether the local repair module is necessary for the overall performance of the closed-loop orchestration framework.

To this end, we compare the full TopoGuard system with an ablated variant that disables repair while keeping the same historical profiles, candidate pool, and test contexts. Table 3 shows that removing repair consistently degrades all three system-level metrics. With repair enabled, TopoGuard achieves higher final quality, lower cost, and lower latency than the variant without repair. This result suggests that local repair does not merely improve isolated failure cases, but also helps the overall system avoid inefficient or suboptimal execution trajectories once intermediate outputs become unreliable.

Notably, the gain of repair is obtained through *bounded local adjustment* rather than global replanning. This is important in practice because it preserves the efficiency of the initial orchestration policy while still providing a corrective mechanism for difficult execution cases. The observed repair trigger rate further suggests that repair is activated only for a minority of cases, indicating that it functions as a targeted safeguard rather than an always-on overhead.

To further understand how repair contributes to system improvement, Fig. 6 provides a detailed analysis of the triggered repair

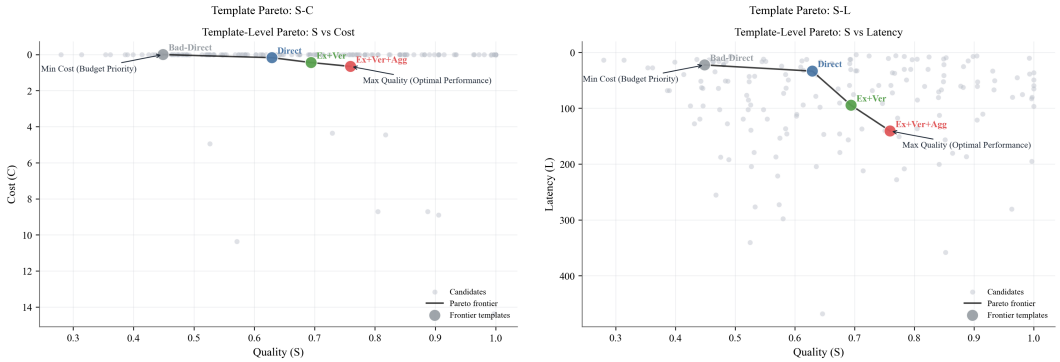

Figure 3: Template-level Pareto frontiers under quality–cost and quality–latency views. Candidate workflow templates exhibit clear non-dominated trade-offs, indicating that topology selection is inherently a multi-objective decision problem.

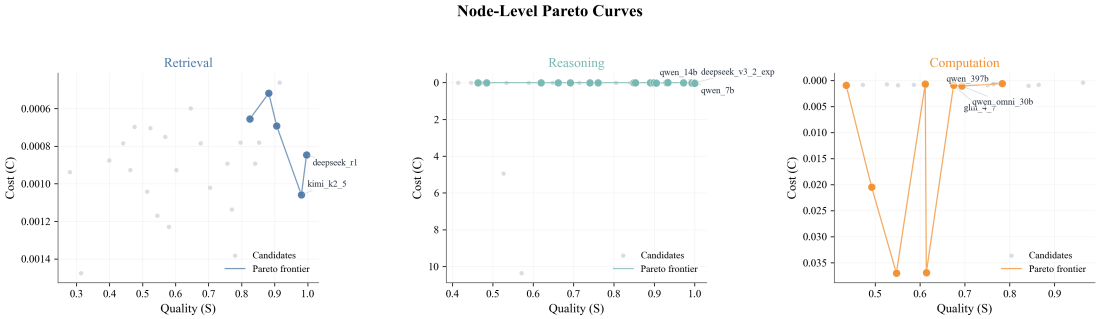

Figure 4: Node-level Pareto curves for different function types. Candidate executors exhibit distinct quality–cost trade-offs across retrieval, reasoning, computation, verification, and aggregation nodes.

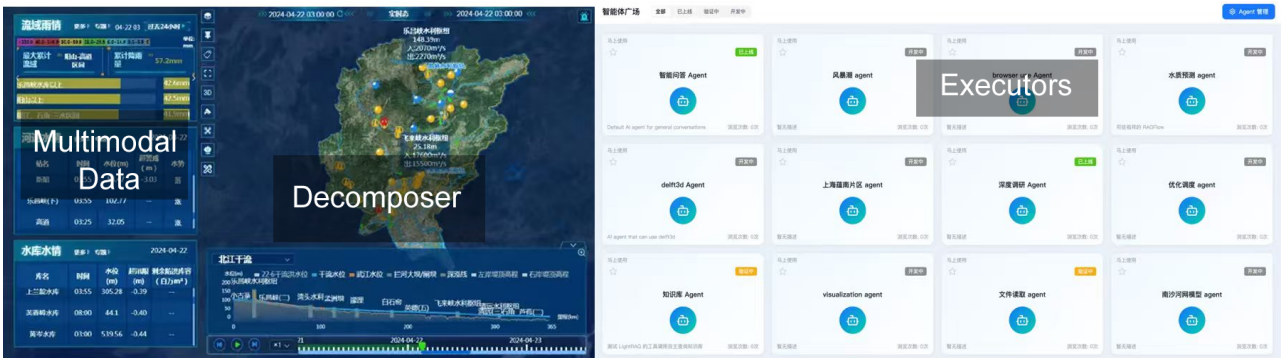

Figure 5: Overview of the water-conservancy digital-twin agent platform used in our case study. The platform integrates multimodal monitoring data, task decomposition, and a pool of available executors, providing the execution environment in which TopoGuard constructs and repairs workflows.

behaviors in Exp 3. Rather than relying on frequent global replanning, TopoGuard mainly uses lightweight local interventions after a repair trigger is detected. This analysis complements the ablation results by showing not only that repair improves overall system performance, but also *how* the improvement is achieved in practice.

Figure 6 reveals three important observations. First, repair is not dominated by heavy structural modifications. As shown in the left panel, most triggered repairs correspond to evaluator-level upgrades, whereas topology-level changes occur less frequently and executor replacement is only occasionally required. This pattern suggests that many execution failures can be corrected through lightweight local adjustment without invalidating the original orchestration decision.

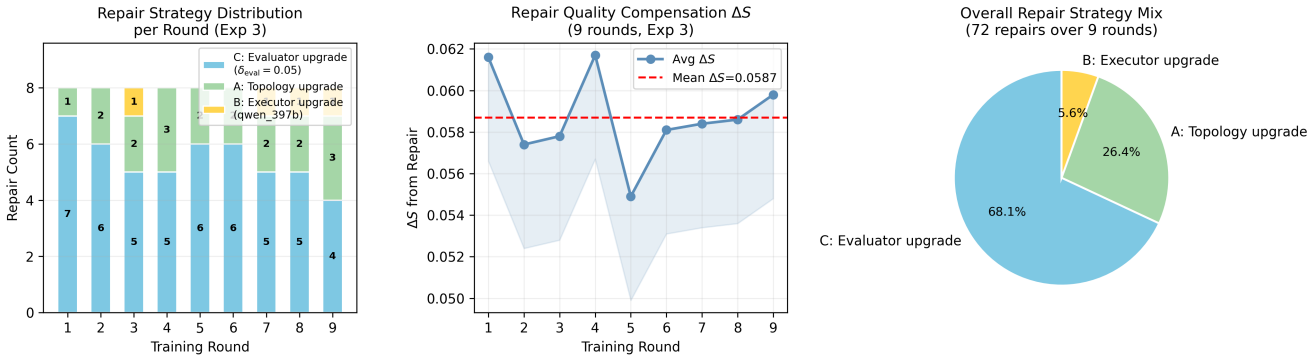

Figure 6: Local repair analysis in Exp 3. Left: repair strategy distribution across rounds. Evaluator upgrade dominates, while topology and executor upgrades are less frequent. Middle: average quality compensation ΔS per round, showing consistent positive gains from repair (mean $\Delta S = 0.0587$). Right: overall repair strategy mix over all rounds. Most repairs are lightweight evaluator-level adjustments, indicating that TopoGuard mainly recovers from intermediate failures through bounded local correction rather than global replanning.

Table 3: Ablation on local repair. Best values are boldfaced.

Variant	Quality $S\uparrow$	Cost (USD) $\downarrow$	Latency (s) $\downarrow$
TopoGuard w/ Repair	**0.7005**	**0.2785**	**76.6**
TopoGuard w/o Repair	0.6914	0.3010	78.8

Additional statistics	Value	Interpretation
Repair trigger rate	14.3% (72/504)	selective activation
Average gain per trigger	+0.059 / trigger	bounded correction benefit

Second, the middle panel shows that repair consistently delivers positive quality compensation across all rounds, with an average gain of 0.0587 per trigger. This indicates that repair is not only useful in a few isolated cases, but provides stable corrective benefit whenever the evaluator detects unreliable intermediate execution states.

Third, the right panel summarizes the overall repair strategy composition over all rounds. The dominance of evaluator upgrades again shows that TopoGuard mainly relies on small and targeted modifications, while more expensive structural interventions remain relatively rare. Taken together, these observations support the central conclusion of this ablation: the effectiveness of TopoGuard comes not only from selecting a strong initial executable topology, but also from its ability to recover efficiently from local failures through bounded repair.

4.6 Sensitivity to Utility Weights

We further study **RQ3** from the utility perspective: whether the conclusions remain stable under different weight settings in the orchestration utility. A desirable property is that changing the weights should induce predictable shifts in topology preference, while the overall advantage of the method remains stable.

To test this, we vary the utility weights (α, β, γ) and re-run candidate scoring on the same held-out contexts without retraining historical profiles. We consider seven representative configurations,

Table 4: Sensitivity analysis under different utility weight settings. The best quality value is boldfaced.

Weight Setting	α	β	γ	$S\uparrow$
Default	0.6	0.2	0.2	0.659
Quality-dominant	0.8	0.1	0.1	0.694
Balanced	0.33	0.33	0.34	0.640
Cost-priority	0.1	0.8	0.1	0.649
Latency-priority	0.1	0.1	0.8	0.613
Q+L only	0.6	0.0	0.4	0.640
Q+C only	0.6	0.4	0.0	**0.699**

including the default setting, quality-dominant weighting, balanced weighting, cost-priority weighting, latency-priority weighting, and two partial-objective settings that remove either cost or latency from the score.

Table 4 shows two clear patterns. First, changing the weights leads to predictable structural shifts in the selected workflows. For example, increasing the quality weight makes the system choose deeper verification- or aggregation-enhanced structures more often, whereas prioritizing latency pushes the policy toward more direct execution paths. Second, the overall conclusion remains stable across most settings: TopoGuard continues to maintain competitive quality and avoids clearly undesirable structures except under the most latency-dominated configuration. This indicates that the advantage of TopoGuard does not hinge on a single hand-tuned utility, but rather arises from the feasible Pareto screening mechanism itself.

The quantitative results show that TopoGuard improves the quality–cost–latency trade-off under constrained execution. To further illustrate how this advantage appears in a realistic deployment setting, we present a case study on a storm-surge risk-warning task. Rather than focusing on aggregate metrics, this case emphasizes the

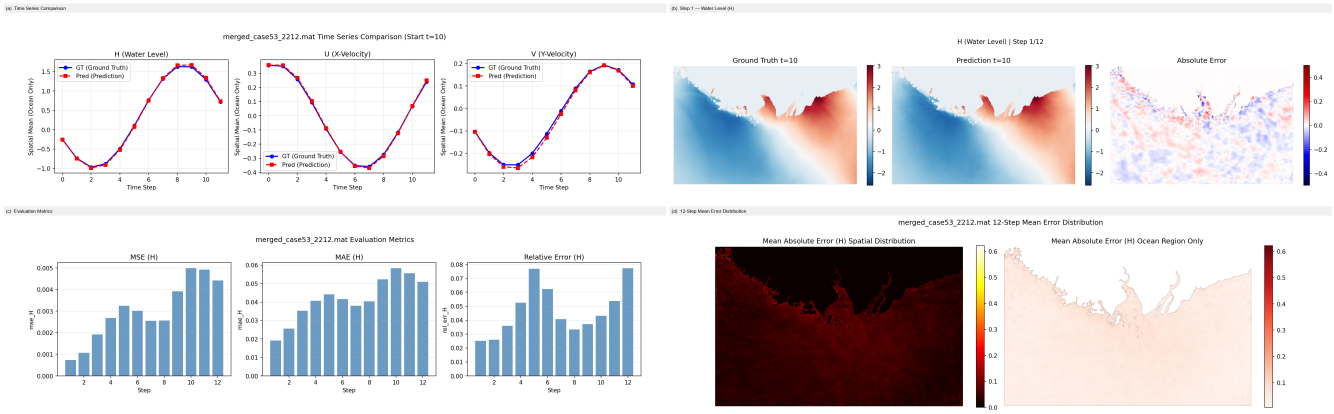

Figure 7: Representative forecast outputs produced by the forecast module in the storm-surge case study, including temporal evolution, key-step spatial comparison, metric statistics, and spatial error distribution. These heterogeneous outputs serve as intermediate evidence for the downstream analysis node.

runtime behavior of the framework, especially how it maintains execution continuity when a downstream node becomes mismatched with the intermediate outputs.

4.7 Case Study: Execution and Local Repair in a Storm-Surge Scenario

We conduct the case study on a water-conservancy digital-twin agent platform, shown in Fig. 5. The platform integrates three key components: multimodal monitoring data, a task decomposition interface, and a pool of executable domain agents. This environment provides a realistic setting for evaluating how TopoGuard organizes available executors into an executable workflow for storm-surge warning tasks.

Given a user request and the available tool set, TopoGuard first generates an initial execution topology and assigns a default downstream analysis node. The forecast module is then invoked to produce intermediate outputs for subsequent interpretation. In our case, these outputs are not simple scalar values, but a collection of heterogeneous artifacts, including temporal evolution curves, spatial comparison results at a key time step, metric summaries, and spatial error maps, as shown in Fig. 7.

These forecast outputs are passed to the downstream analysis node as intermediate evidence. However, the default analysis node fails to adequately interpret this heterogeneous output package and therefore cannot produce a satisfactory downstream result. Instead of replanning the entire workflow, TopoGuard triggers bounded local repair and replaces only the current analysis node, while preserving the forecast outputs generated upstream.

After repair, the system resumes execution using the same forecast evidence and continues the subsequent warning-generation process. This case study illustrates two practical properties of TopoGuard: it can construct an executable initial topology in a realistic multimodal platform, and it can repair local runtime mismatches without discarding already-computed upstream results.

4.8 Discussion

Two observations are worth highlighting. First, TopoGuard is not designed to maximize quality alone, but to identify better operating points under joint quality, cost, and latency constraints. Therefore, the fact that **Best-Quality** may achieve slightly higher quality in some cases does not weaken our claim; instead, it highlights the practical value of balanced topology orchestration in deployment.

Second, local repair should be viewed as a bounded safeguard rather than a replacement for initial planning. The initial orchestration policy handles most normal cases effectively, while repair provides targeted correction for failure-prone executions. This improves system resilience without the overhead of repeated global replanning.

Overall, the results support the central claim of this paper: **adaptive topology orchestration, formulated as joint workflow selection, executor assignment, and bounded local repair, yields a stronger quality–cost–latency trade-off than single-preference orchestration strategies.**

5 Conclusion

This paper studied adaptive topology orchestration for complex multi-stage task solving under joint quality–cost–latency constraints. We argue that the key challenge is not merely selecting a strong executor, but constructing and maintaining an executable, efficient, and reliable collaboration topology among heterogeneous capability units, especially when task structures vary and intermediate outputs may be unreliable.

To address this problem, we presented TopoGuard, a closed-loop orchestration framework that unifies workflow-level topology selection, node-level executor assignment, and bounded local repair within a constrained multi-objective decision process. By modeling topology as the optimization target, TopoGuard enables targeted adaptation without expensive global replanning.

Experiments on a water-conservancy digital-twin platform show that TopoGuard achieves a better overall trade-off among quality, cost, and latency than representative alternatives. Further analyses

show that this advantage is supported by the Pareto structure of workflow templates and node executors, remains stable under different utility preferences, and transfers across task domains. The ablation study also confirms that local repair provides an effective safeguard in difficult execution cases.

Overall, these results suggest that topology-aware closed-loop orchestration provides a practical foundation for adaptive multi-stage systems under real deployment constraints.

References

[1] Nilani Algiriyage, Raj Prasanna, Kristin Stock, Emma EH Doyle, and David Johnston. 2022. Multi-source multimodal data and deep learning for disaster response: a systematic review. *SN Computer Science* 3, 1 (2022), 92.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176* (2023).

[3] Ruijie Cheng, Lei Hou, and Sheng Xu. 2023. A review of digital twin applications in civil and infrastructure emergency management. *Buildings* 13, 5 (2023), 1143.

[4] Joris Dekoninck and coauthors. 2025. A Unified Approach to Routing and Cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*.

[5] Shengda Fan, Xin Cong, Yuepeng Fu, Zhong Zhang, Shuyan Zhang, Yuanwei Liu, Yesai Wu, Yankai Lin, Zhiyuan Liu, and Maosong Sun. 2024. Workflowllm: Enhancing workflow orchestration capability of large language models. *arXiv preprint arXiv:2411.05451* (2024).

[6] Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. 2024. Understanding the planning of llm agents: A survey. *arXiv preprint arXiv:2402.02716* (2024).

[7] Solomon Inyang and Firouzeh Rosa Taghikhah. 2025. Digital twin-based decision support systems for natural disaster management: A systematic review of current trends and approaches. *Science Talks* (2025).

[8] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An llm compiler for parallel function calling. In *Forty-first International Conference on Machine Learning*.

[9] Guilin Lan, Yeqian Du, and Zhouwang Yang. 2024. Robust multimodal representation under uncertain missing modalities. *ACM Transactions on Multimedia Computing, Communications and Applications* 21, 1 (2024), 1–23.

[10] Guilin Lan, Ye-Qian Du, and Zhouwang Yang. 2025. Robust Multimodal Representation under Uncertain Missing Modalities. *ACM Transactions on Multimedia Computing, Communications, and Applications* 21, 1 (2025), 34:1–34:23.

[11] LangChain. 2026. LangGraph Documentation: Workflows and Agents. https://docs.langchain.com/oss/python/langgraph/workflows-agents. Accessed 2026-04-01.

[12] Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. Autoflow: Automated workflow generation for large language model agents. *arXiv preprint arXiv:2407.12821* (2024).

[13] Muhammad Irzam Liaqat, Shah Nawaz, Muhammad Zaigham Zaheer, Muhammad Saad Saeed, Hassan Sajjad, Tom De Schepper, Karthik Nandakumar, Muhammad Haris Khan, Ignazio Gallo, and Markus Schedl. 2025. Chameleon: A multimodal learning framework robust to missing modalities. *International Journal of Multimedia Information Retrieval* 14, 2 (2025), 21.

[14] Ronghao Lin and Haifeng Hu. 2023. Missmodal: Increasing robustness to missing modality in multimodal sentiment analysis. *Transactions of the Association for Computational Linguistics* 11 (2023), 1686–1702.

[15] Microsoft. 2024. Semantic Kernel Process Framework. https://learn.microsoft.com/en-us/semantic-kernel/frameworks/process/process-framework. Accessed 2026-04-01.

[16] Drashti Pathak, Aparna S. Varde, Gerard de Melo, and Clement Alo. 2020. Hydroinformatics and the Web: Analytics and Dissemination of Hydrology Data for Climate Change and Sustainability. *SIGWEB Newsletter* 2020, Autumn (2020), 3:1–3:17.

[17] Md Kaykobad Reza, Ashley Prater-Bennette, and M Salman Asif. 2024. Robust multimodal learning with missing modalities via parameter-efficient adaptation. *IEEE transactions on pattern analysis and machine intelligence* 47, 2 (2024), 742–754.

[18] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems* 36 (2023), 68539–68551.

[19] Mengfan Tang, Siripen Pongpaibool, and Ramesh C. Jain. 2016. Research Challenges in Developing Multimedia Systems for Managing Emergency Situations. In *Proceedings of the 24th ACM International Conference on Multimedia*. 938–947.

[20] Arkadiusz Tomczak, Grzegorz Stępień, Tomasz Kogut, Łukasz Jedynak, Grzegorz Zaniewicz, Małgorzata Łącka, and Izabela Bodus-Olkowska. 2024. Development of a Digital Twin of the Harbour Waters and Surrounding Infrastructure Based on Spatial Data Acquired with Multimodal and Multi-Sensor Mapping Systems. *Applied Sciences* 15, 1 (2024), 315.

[21] Renjie Wu, Hu Wang, Hsiang-Ting Chen, and Gustavo Carneiro. 2024. Deep multimodal learning with missing modality: A survey. *arXiv preprint arXiv:2409.07825* (2024).

[22] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[23] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[24] Xiaoya Yu, Kejun Wu, You Yang, and Qiong Liu. 2024. WaRENet: A Novel Urban Waterlogging Risk Evaluation Network. *ACM Transactions on Multimedia Computing, Communications, and Applications* 20, 7 (2024), 214:1–214:28.

[25] Zhen Zeng, William Watson, Nicole Cho, Saba Rahimi, Shayleen Reynolds, Tucker Balch, and Manuela Veloso. 2023. FlowMind: automatic workflow generation with LLMs. In *Proceedings of the Fourth ACM International Conference on AI in Finance*. 73–81.

[26] Wenshuo Zhai, Jinzhi Liao, Ziyang Chen, Bolun Su, and Xiang Zhao. 2025. A survey of task planning with large language models. *Intelligent Computing* 4 (2025), 0124.

[27] Duzhen Zhang, Yahan Yu, Jiahua Dong, Chenxing Li, Dan Su, Chenhui Chu, and Dong Yu. 2024. Mm-llms: Recent advances in multimodal large language models. *Findings of the Association for Computational Linguistics: ACL 2024* (2024), 12401–12430.

[28] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. 2024. Aflow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762* (2024).